

报告二：基于Appium自动化的长时间压力性能测试报告

一、测试环境与文件说明

1.1 环境信息

配置项	详细信息
自动化框架	Appium Python Client 2.11.1
执行解释器	D:\Tools\python.exe
被测设备	emulator-5554 Pixel7 API34
自动化脚本	appium_perf_test.py （自动化操作+adb性能采集）
绘图脚本	plot_results.py （读取CSV日志生成内存趋势图）

1.2 项目文件清单及用途

文件名称	文件作用
appium_perf_test.py	核心自动化脚本，实现3类压力场景循环操作，每10秒调用adb采集内存、帧率数据
01_滑动列表_性能日志.csv	场景1：持续滑动账单列表3分钟性能数据
02_页面切换_性能日志.csv	场景2：循环「首页-新增账单页」来回跳转3分钟性能数据
03_增删账单_性能日志.csv	场景3：循环批量新增5条账单后删除，3分钟高频读写加压日志
04_空闲监控_性能日志.csv	后台静置空闲对照组，采集无操作时内存基线
plot_results.py	读取四份CSV日志，Matplotlib绘制内存时序折线图
performance_full_report.png	最终生成的内存对比趋势可视化图表

二、三大自动化压力场景测试流程

场景1：账单列表持续滑动压力测试

项目	说明
自动化逻辑	模拟器自动循环上下滑动账单列表，单次滑动间隔0.5s，持续加压3min
数据采集	每10s执行 <code>dumpsys meminfo</code> 采集应用PSS内存
输出文件	<code>01_滑动列表_性能日志.csv</code>
数据表现	内存基线平稳小幅震荡，仅滑动加载新列表项瞬时抬升，GC后快速回收

场景2：页面循环切换压力测试

项目	说明
自动化逻辑	自动点击「记一笔」跳转新增账单页，1s后返回首页，无限循环3min
输出文件	<code>02_页面切换_性能日志.csv</code>
数据表现	每次页面重建会创建新Compose宿主，页面栈对象回收存在延迟，内存基线小幅抬升

场景3：批量增删账单高频读写压力测试

项目	说明
自动化逻辑	每轮自动新增5条账单数据，完成后模拟批量删除，循环执行3min
输出文件	<code>03_增删账单_性能日志.csv</code>
核心现象	内存整体震荡上行，每轮操作峰值逐级抬高，GC无法完全释放临时数据与视图对象

场景4：空闲对照组

项目	说明
操作	应用停留在首页无任何操作，采集3min空闲内存
输出文件	<code>04_空闲监控_性能日志.csv</code>
作用	作为基准参考

三、ComposeApplication 全维度性能可视化图表数据分析

图表整体说明

图表共6张子图，覆盖**内存、CPU、帧率、卡顿率、电池温度、场景均值对比**6大性能维度，4组测试场景：

场景标识	颜色	说明
滑动列表	蓝色	持续上下滑动账单列表
页面切换	橙色	循环首页↔新增账单页跳转
增删账单	绿色	批量新增+删除账单高频读写
空闲监控	红色	App静置后台无操作（基准对照组）

译

四、分图逐项解析

4.1 左上：内存使用趋势对比（单位MB）

场景	表现	结论
空闲基准（红线）	稳定维持119~120MB	应用基础常驻内存
滑动列表（蓝线）	启动冲高至123MB，后回落至121~122MB	仅列表加载时临时占用，GC可回收
页面切换（橙线）	启动峰值125MB，全程基线121~123MB缓慢抬升	页面重建带来持续内存堆积
增删账单（绿线）	全场景内存最高，稳定124.5~125MB	存在渐进式内存泄漏风险

4.2 右上：CPU使用率趋势对比（单位%）

场景	CPU表现	结论
空闲监控（红线）	稳定10%~12%	最低CPU负载
滑动列表（蓝线）	稳定13%~14%	列表渲染、图片解码持续占用
页面切换/增删账单	前期13%，后期降至11%	后期缓存生效负载回落

阈值参考：绿色虚线30%为高负载警戒线，所有场景CPU均远低于警戒线，**无CPU过载卡顿。**

4.3 左中：帧率FPS趋势对比

场景	FPS表现	结论
空闲监控	稳定60满帧	无UI渲染压力
滑动列表	全程58~60FPS	接近满帧，流畅
页面切换/增删账单	后期下跌，最低趋近30FPS	页面重建、数据刷新阻塞主线程

标准参考：红色虚线30FPS为流畅临界线，绿色虚线60为满帧标准。

4.4 右中：Jank卡顿率对比

场景	Jank率	结论
滑动列表、空闲监控	几乎为0	无明显卡顿帧
页面切换、增删账单	采样后半段微小波动	整体卡顿风险低

4.5 左下：电池温度变化（单位℃）

- 四条曲线完全重合，全程稳定25℃
- 各场景CPU、GPU负载均较低，不会引发设备发热、降频
- 功耗控制合格

4.6 右下：各场景平均性能柱状对比

场景	平均内存(MB)	平均CPU(%)
滑动列表	122	12.2
页面切换	123	11.0
增删账单	125	11.0
空闲监控	120	11.0

横向规律：

- 内存排序：增删账单 > 页面切换 > 滑动列表 > 空闲监控
- CPU排序：滑动列表 > 页面切换 = 增删账单 = 空闲监控

五、综合性能问题总结

5.1 现存性能缺陷

问题类型	严重程度	具体表现
内存泄漏	🔴 核心问题	页面切换、增删账单场景内存持续走高，GC无法回收至空闲基准
帧率衰减	🟡 中等问题	频繁页面跳转、数据读写阻塞主线程，后期帧率逼近30FPS

无严重风险点：CPU未过载、无明显Jank卡顿、电池无升温。

5.2 分场景优化优先级

优先级	场景	优化方案
★ 最高	增删账单	复用列表数据对象，操作后清空临时数据集
★★ 次高	页面跳转	使用Compose导航复用页面实例，避免重复重建
★★★ 常规	数据库操作	批量读写异步化，移入协程后台



六、Appium自动化测试优缺点总结

优势

优势	说明
无人值守长时间加压	可模拟连续高频操作，复现渐进式内存泄漏
数据自动持久化	CSV时序完整可量化，支持批量绘图
标准化可复现	便于优化前后对比回归

局限

局限	说明
指标粒度较粗	仅采集全局内存、宏观帧率，无法精准定位具体泄漏对象
操作精度有限	ADB指令延迟，精度低于人工Profiler调试